UAV Track Write-up

UAV-H1: Dead Reckoning

1. What I tried first, and why it did not work

My first idea was to plot the GPS fixes on a map and look for where the flight path started to look wrong. That did not work, because the spoofing was designed to be subtle. The fake GPS drifted the drone slowly with no obvious jumps, so the map alone could not tell me when the attack began.

2. The insight that actually cracked it

GPS can be spoofed remotely, but the drone's IMU (gyros and accelerometers) measures real physical motion and can't be faked that way. I wrote a script using SciPy to integrate the IMU's raw acceleration over time and get the drone's actual velocity. The IMU had sensor drift, so I detrended the data first. Then I subtracted the IMU velocity from the GPS velocity. The difference stayed close to zero until one specific timestamp, where it started ramping up steeply. That timestamp is when the spoofing began. I then used atan2 on the final offset values to get the compass bearing.

3. How I satisfied myself the answer was right

I plotted the difference between the IMU and GPS velocities. Instead of random noise, it was a flat line that turned into a steep ramp at the timestamp I had calculated. A clean break like that is what a spoofing offset starting at a single moment should look like, so I was confident I had isolated it.

4. Where I used an AI assistant and what I had to correct

I knew I had to integrate the acceleration, but I had not dealt with raw sensor drift before. I asked an AI why my integrated velocity kept growing without bound, and it explained IMU bias and detrending. It suggested using np.polyfit on the early part of the flight, before the attack, to fit the drift slope and then subtract it. That worked without changes.

UAV-H2: Hopscotch 2

1. What I tried first, and why it did not work

I first thought about reversing the LFSR math from the partial channel log, by setting up a system of equations to solve for the seed. I dropped that quickly, because the log was too messy for it. It had noise, false detections and missing channels, and any strict set of equations would break on the first bad entry.

2. The insight that actually cracked it

A 20-bit LFSR only has 1,048,576 possible starting states, which is small enough to brute-force. I wrote a script that generates the channel sequence for every possible 20-bit seed. For each seed, I counted how many of its predicted channels matched the timestamps in the log. The correct seed would overlap heavily with the log, and since I was counting matches instead of requiring a perfect fit, the noise did not matter.

3. How I satisfied myself the answer was right

Almost every seed matched the log only 5 or 6 times, which is what chance would give. One seed matched more than 30 times. A gap that large is far outside what random matching would produce, so I took that seed as the correct one.

4. Where I used an AI assistant and what I had to correct

I used an AI to write the core lfsr_step function. The XOR taps for a Fibonacci LFSR need specific bitwise operations that are easy to get wrong in Python. I gave it the tap indices from the challenge description and it returned a fast bit-shift version. I wrote the brute-force search loop myself.

UAV-H3: Consensus

1. What I tried first, and why it did not work

I could not just feed all 6 sensor timestamps into a standard Time Difference of Arrival (TDOA) solver. The challenge said exactly 2 of the sensors were compromised and reporting false data, and a standard solver breaks down when any input is bad. Using all six would have given coordinates far from the port.

2. The insight that actually cracked it

The real problem was working out which 4 sensors were honest. There are 15 ways to choose 4 sensors out of 6, so I wrote a script to test every subset. For each group of 4, I ran a Nelder-Mead optimizer to find the drone position with the smallest TDOA error. The 14 groups that included at least one lying sensor all had large errors. Only one group had an error close to zero. That group gave me the drone's position, and the 2 sensors left out of it were the compromised ones.

3. How I satisfied myself the answer was right

I checked the error values. For the wrong groups, the optimizer could not get the error below several thousand meters. For the correct group of 4, it dropped to about 0.0001. That is a very large difference, and it shows those 4 sensors were consistent with a single physical position, while no other combination was.

4. Where I used an AI assistant and what I had to correct

I had not heard of the Nelder-Mead algorithm or error minimization before this challenge. Once I knew I had to test combinations of sensors, I asked an AI how to find coordinates from the noisy TDOA data. It explained objective functions and showed me how to use scipy.optimize.minimize with Nelder-Mead to search for the location. I only had to put my distance calculations into the objective format it gave me.